Identity and Access Management (IAM)

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

NORTHEASTERN.

Learning Objectives

- Explain what Identity and Access Management (IAM) is and why it is critical in cloud environments
- Describe the principle of least privilege and how it applies to cloud resource access
- Identify the core IAM components across AWS, GCP, and Azure
- Compare and contrast IAM models across the three major cloud providers

What Is Cloud IAM

- The problem: in the cloud, resources are accessible over the network — who gets access to what?
- IAM defined (Gartner): "the security discipline that enables the right individuals to access the right resources at the right times for the right reasons"
- Two fundamental questions IAM answers:
  - **Authentication** — "Who are you?" (proving identity)
  - **Authorization** — "What are you allowed to do?" (granting permissions)
- **Secure by default**: in cloud IAM, users have no access to resources until permissions are explicitly granted (deny by default)
- The **Principle of Least Privilege**: grant only the minimum permissions necessary to perform a task
- Why IAM matters: misconfigured access is one of the most common causes of cloud security breaches

Core IAM Concepts

- **Identities (Principals)**: the entities that can request access — users, groups, service accounts, applications
- **Policies / Permissions**: rules that define what actions an identity can perform on which resources
- **Roles**: collections of permissions that can be assigned to identities — avoids managing permissions individually
- **Groups**: collections of users that share the same set of permissions
- **Service Accounts / Machine Identities**: identities used by applications and services (not humans) to interact with cloud APIs
- **Credential types**: passwords, access keys, key pairs, X.509 certificates — different credential types serve different authentication methods (console login, API access, programmatic access)
- **Multi-Factor Authentication (MFA)**: requiring more than one form of verification to strengthen authentication
- **Identity Federation**: leveraging external identity systems (corporate directories like Active Directory, or web identity providers like Google) so users can access cloud resources with existing credentials rather than creating separate cloud accounts
- The **Root Account / Organization Admin** problem: overly powerful accounts that should be locked down and rarely use

Key Benefits of Cloud IAM

- **Enhanced security**: granular control over who can access what, secure by default
- **Shared access without sharing credentials**: grant others permission to use resources in your cloud account without sharing passwords or access keys
- **Temporary credentials via roles**: authenticated users or services can assume roles to get time-limited access, reducing risk from long-lived credentials
- **Flexible credential management**: multiple authentication methods depending on use case (console, CLI, API)
- **Integration with external identity systems**: federate with corporate directories or web identity providers
- **Seamless integration across cloud services**: IAM policies apply uniformly across all services within a cloud provider

AWS IAM

AWS IAM

- AWS IAM is a **global service** — not region-specific
- AWS IAM is **secure by default** — new users have no access to AWS resources until permissions are explicitly granted
- IAM is seamlessly integrated into most AWS services — access controls defined in one place apply throughout the AWS environment

Key components

- **IAM Users**: individual identities with long-term credentials (username/password, access keys)
- **IAM Groups**: collections of IAM users that share permissions (e.g., "Developers," "Admins")
- **IAM Roles**: temporary credential sets that can be assumed by users, services, or other AWS accounts — preferred over long-term credentials
- **IAM Policies**: JSON documents that define Allow/Deny rules for actions on resources

Key features

- Policy structure: **Shared access**: grant others permission to administer and use resources without sharing passwords or access keys
- **Granular permissions**: grant different permissions to different people for different resources (e.g., terminate EC2 instances, read S3 bucket contents)
- **Secure access for EC2 applications**: use IAM to securely provide applications running on EC2 with credentials to access other AWS resources (via instance profiles / roles)
- **Flexible credential management**: passwords, access keys, key pairs, X.509 certificates, MFA
- **Identity federation**: grant access using existing identity systems — supports federation from corporate systems (Microsoft Active Directory) and web identity providers (Google, Facebook)
- Effect (Allow/Deny), Action (e.g., s3:GetObject), Resource (ARN specifying which resource)

Best practices

- Lock away your AWS account root user access keys
- Create individual IAM users (never share credentials)
- Use AWS-defined (managed) policies to assign permissions whenever possible
- Use groups to assign permissions to IAM users
- Grant least privilege
- Use access levels to review IAM permissions
- Configure a strong password policy for your users
- Enable MFA for privileged users
- Use roles for applications that run on Amazon EC2 instances
- Delegate by using roles instead of by sharing credentials
- Rotate credentials regularly
- Remove unnecessary credentials

GCP IAM

GCP IAM

- GCP IAM is built around a **resource hierarchy**: Organization → Folders → Projects → Resources
- Permissions **inherit downward** through the hierarchy — a role granted at the project level applies to all resources within that project
- Key components:
  - **Google Accounts & Groups**: human identities managed through Google Workspace or Cloud Identity
  - **Service Accounts**: identities for applications and VMs — also act as resources that can have their own policies
  - **Roles**: three types:
    - **Basic Roles**: Owner, Editor, Viewer (broad — avoid in production)
    - **Predefined Roles**: fine-grained, provider-managed (e.g., roles/storage.objectViewer)
    - **Custom Roles**: user-defined combinations of specific permissions
  - **IAM Policy Bindings**: attach a role to a member (user, group, or service account) on a specific resource
- Best practices:
  - Prefer predefined roles over basic roles
  - Bind roles at the narrowest appropriate level of the resource hierarchy
  - Use service accounts for application-to-application authentication
  - Leverage Workload Identity Federation to avoid exporting service account keys

Azure IAM

Azure IAM

- Azure uses **Microsoft Entra ID** (formerly Azure Active Directory / Azure AD) as its identity provider
- Access control is managed through **Azure Role-Based Access Control (RBAC)** applied to a resource hierarchy: Management Groups → Subscriptions → Resource Groups → Resources
- Key components:
  - **Users & Groups**: identities managed in Microsoft Entra ID — supports integration with on-premises Active Directory
  - **Service Principals & Managed Identities**: application identities — Managed Identities (system-assigned or user-assigned) are Azure's recommended approach for service-to-service authentication, eliminating the need to manage credentials
  - **Roles**: built-in roles (Owner, Contributor, Reader, plus many fine-grained roles) or custom roles
  - **Role Assignments**: the combination of a security principal + a role definition + a scope (resource level)
- Best practices:
  - Use Managed Identities wherever possible for Azure service-to-service access
  - Assign roles at the most specific scope needed
  - Use Microsoft Entra groups for role assignments rather than individual users
  - Enable Conditional Access policies for context-aware access decisions

Cross-Provider Comparison Summary

Concept	AWS	GCP	Azure
Identity Provider	AWS IAM	Google Cloud Identity	Microsoft Entra ID
Human Identities	IAM Users	Google Accounts	Entra ID Users
Machine Identities	IAM Roles / Instance Profiles	Service Accounts	Managed Identities / Service Principals
Grouping	IAM Groups	Google Groups	Entra ID Groups
Permissions Model	JSON Policies	Role Bindings on Resources	RBAC Role Assignments
Resource Hierarchy	Account (→ Organizations)	Org → Folders → Projects	Mgmt Groups → Subscriptions → Resource Groups
Permission Inheritance	Policies attached directly	Inherits down hierarchy	Inherits down hierarchy
Identity Federation	AWS IAM Identity Center, SAML, OIDC	Workload Identity Federation, Google Cloud Identity	Microsoft Entra External ID, SAML, OIDC
Default Access	Deny all (secure by default)	Deny all (secure by default)	Deny all (secure by default)

Summary

Key Takeaways

- Every cloud provider solves the same core problem — who can do what to which resources — but with different terminology and structural models
- Understanding one provider's IAM deeply makes it much easier to learn the others

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS